Exercise 3: The numerical toolbox

Prognostic Modelling (pea700), WiSe 2026/27

Out: week 3 · Back: week 4 · 13 points

Covers unit 3 of the script. Problem 1 is worked on in the lab; problems 2 to 4 are homework.

Your toolbox. From this sheet on, collect the functions you write that are useful beyond one problem, such as the quadrature rules, bisection and Newton's method, in a file `toolbox.py` in your repository. In a later sheet, or in your project, you then write `from toolbox import bisection` and use your own, tested function instead of writing it again.

Problem 1: Truncation and round-off error (in the lab, 4 points)

Approximate the derivative of $u(x) = e^{-x^2}$ at $x = 0.5$ with the forward and the central difference.

a) Plot the absolute error of both as a function of h for $h = 10^{-1}, \dots, 10^{-14}$ with logarithmic axes.

b) Determine the observed order of both formulas in a range of h where round-off plays no role.

c) Read the optimal step and the smallest error from your plot and compare them with the estimates $h \sim \sqrt{\varepsilon}$ and $h \sim \varepsilon^{1/3}$ from the script.

Solution

The code is given twice: step by step, as you would write it, and in a compact form that works with whole NumPy arrays. Both give the same results.

The derivatives. For the error we need the exact derivative, and for c) also the second and third derivative. We write $u = e^g$ with $g(x) = -x^2$, $g'(x) = -2x$. By the chain rule, $\frac{\mathrm{d}}{\mathrm{d}x} e^g = g'\, e^g$, so

$$u'(x) = -2x\,e^{-x^2}.$$

For u'' we differentiate the product $(-2x)\cdot e^{-x^2}$ with the product rule, $(pq)' = p'q + pq'$, where $p = -2x$, $p' = -2$ and $q = e^{-x^2}$, $q' = -2x\,e^{-x^2}$:

$$u''(x) = (-2)\,e^{-x^2} + (-2x)(-2x)\,e^{-x^2} = -2\,e^{-x^2} + 4x^2\,e^{-x^2} = (4x^2 - 2)\,e^{-x^2}.$$

The same rule with $p = 4x^2 - 2$, $p' = 8x$:

$$u'''(x) = 8x\,e^{-x^2} + (4x^2 - 2)(-2x)\,e^{-x^2} = \left(8x - 8x^3 + 4x\right)e^{-x^2} = (12x - 8x^3)\,e^{-x^2}.$$

At $x = 0.5$ we have $x^2 = 0.25$, $x^3 = 0.125$ and $e^{-0.25} = 0.7788$, so

$$\begin{aligned} u &= 0.7788, \\ u' &= -2 \cdot 0.5 \cdot 0.7788 = -0.7788, \\ u'' &= (4 \cdot 0.25 - 2) \cdot 0.7788 = (1 - 2) \cdot 0.7788 = -0.7788, \\ u''' &= (12 \cdot 0.5 - 8 \cdot 0.125) \cdot 0.7788 = (6 - 1) \cdot 0.7788 = 3.894. \end{aligned}$$

The observed order. If the error behaves as $e(h) = Ch^p$, then for two steps h_1 and h_2

$$\frac{e(h_1)}{e(h_2)} = \frac{Ch_1^p}{Ch_2^p} = \left(\frac{h_1}{h_2}\right)^p \quad\Longrightarrow\quad \ln\frac{e(h_1)}{e(h_2)} = p\,\ln\frac{h_1}{h_2} \quad\Longrightarrow\quad p = \frac{\ln\left(e(h_1)/e(h_2)\right)}{\ln(h_1/h_2)}.$$

The unknown constant C cancels. This is the formula in the code.

Step by step

```
import numpy as np                  # NumPy: np.exp, np.log, ...
import matplotlib.pyplot as plt     # plotting

def u(x):
    return np.exp(-x**2)

def dudx(x):                        # exact derivative
    return -2 * x * np.exp(-x**2)

x = 0.5
exact = dudx(x)

h_values = []
```

```
error_forward = []
error_central = []
for k in range(53):                 # k = 0, ..., 52
    h = 10.0 ** (-1 - 0.25 * k)     # h = 10^-1, 10^-1.25, ..., 10^-14
    forward = (u(x + h) - u(x)) / h
    central = (u(x + h) - u(x - h)) / (2 * h)
    h_values.append(h)
    error_forward.append(abs(forward - exact))
    error_central.append(abs(central - exact))

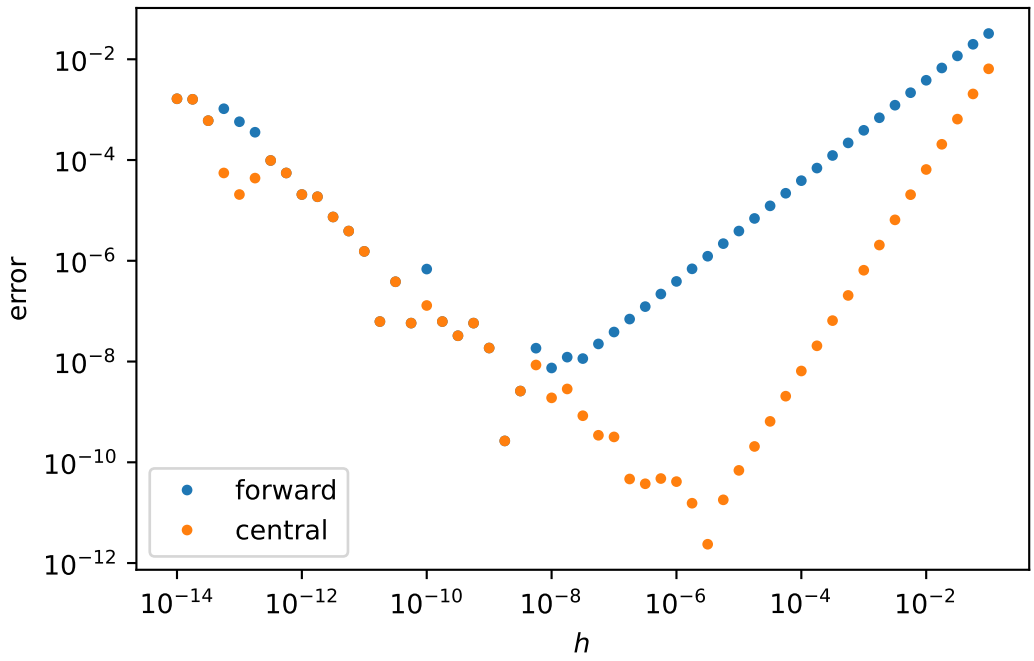
# a) the plot
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.loglog(h_values, error_forward, ".", label="forward")
ax.loglog(h_values, error_central, ".", label="central")
ax.set_xlabel("$h$")
ax.set_ylabel("error")
ax.legend()
plt.show()

# b) observed order from h = 1e-2 and h = 5e-3, far from the round-off range
h1 = 1e-2
h2 = 5e-3
e1_f = abs((u(x + h1) - u(x)) / h1 - exact)
e2_f = abs((u(x + h2) - u(x)) / h2 - exact)
e1_c = abs((u(x + h1) - u(x - h1)) / (2 * h1) - exact)
e2_c = abs((u(x + h2) - u(x - h2)) / (2 * h2) - exact)
p_f = np.log(e1_f / e2_f) / np.log(h1 / h2)
p_c = np.log(e1_c / e2_c) / np.log(h1 / h2)
print(f"observed order, forward: {p_f:.3f}")
print(f"observed order, central: {p_c:.3f}")

# c) smallest error and the step where it occurs, found with a loop
i_best_f = 0                        # position of the smallest error so far
i_best_c = 0
for i in range(len(h_values)):
    if error_forward[i] < error_forward[i_best_f]:
        i_best_f = i
    if error_central[i] < error_central[i_best_c]:
        i_best_c = i
print(f"forward: smallest error {error_forward[i_best_f]:.1e}"
      f" at h = {h_values[i_best_f]:.1e}")
print(f"central: smallest error {error_central[i_best_c]:.1e}"
      f" at h = {h_values[i_best_c]:.1e}")

# c) the estimates of the script
eps = 2.0**-52                      # machine epsilon
u0 = abs(u(x))                      # |u|, |u''|, |u'''| at x = 0.5
u2 = abs((4 * x**2 - 2) * np.exp(-x**2))
```

```
u3 = abs((12 * x - 8 * x**3) * np.exp(-x**2))
h_f = 2 * np.sqrt(eps * u0 / u2)    # forward: h_opt = 2 sqrt(eps |u| / |u''|)
e_f = 2 * np.sqrt(eps * u0 * u2)    #          e = 2 sqrt(eps |u| |u''|)
h_c = (3 * eps * u0 / u3) ** (1 / 3)  # central: h_opt = (3 eps |u| / |u'''|)^(1/3)
e_c = h_c**2 * u3 / 2               #          e = h_opt^2 |u'''| / 2
print(f"estimate forward: h_opt = {h_f:.1e}, error {e_f:.1e}")
print(f"estimate central: h_opt = {h_c:.1e}, error {e_c:.1e}")
```




```
observed order, forward: 0.988
observed order, central: 2.000
forward: smallest error 2.7e-10 at h = 1.8e-09
central: smallest error 2.4e-12 at h = 3.2e-06
estimate forward: h_opt = 3.0e-08, error 2.3e-08
estimate central: h_opt = 5.1e-06, error 5.1e-11
```

Compact

```
import numpy as np
import matplotlib.pyplot as plt

u = lambda x: np.exp(-x**2)                  # a function in one line
dudx = lambda x: -2 * x * np.exp(-x**2)
```

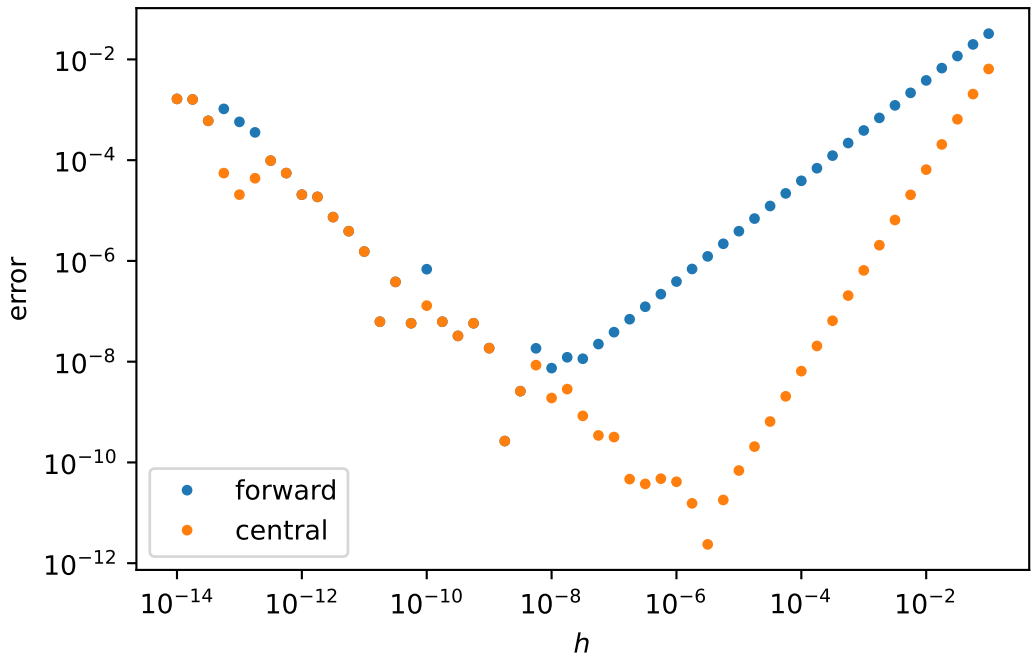
```
x = 0.5

h = 10.0 ** np.arange(-14, -0.9, 0.25)      # all step sizes as one array
error_f = abs((u(x + h) - u(x)) / h - dudx(x))
error_c = abs((u(x + h) - u(x - h)) / (2 * h) - dudx(x))

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.loglog(h, error_f, ".", label="forward")
ax.loglog(h, error_c, ".", label="central")
ax.set(xlabel="$h$", ylabel="error")
ax.legend()
plt.show()

# observed order from neighbouring steps between h = 1e-2 and 2.5e-3
h_test = np.array([1e-2, 5e-3, 2.5e-3])
e_f = abs((u(x + h_test) - u(x)) / h_test - dudx(x))
e_c = abs((u(x + h_test) - u(x - h_test)) / (2 * h_test) - dudx(x))
print("observed order, forward:", (np.log(e_f[:-1] / e_f[1:]) / np.log(2)).round(3))
print("observed order, central:", (np.log(e_c[:-1] / e_c[1:]) / np.log(2)).round(3))

# smallest errors; argmin gives the position of the smallest element
i_f = error_f.argmin()
i_c = error_c.argmin()
print(f"forward: smallest error {error_f[i_f]:.1e} at h = {h[i_f]:.1e}")
print(f"central: smallest error {error_c[i_c]:.1e} at h = {h[i_c]:.1e}")
```



```
observed order, forward: [0.988 0.994]
observed order, central: [2. 2.]
forward: smallest error 2.7e-10 at h = 1.8e-09
central: smallest error 2.4e-12 at h = 3.2e-06
```

a) The errors decrease with slopes 1 and 2 until round-off takes over; then they increase again and scatter.

b) The observed orders are 0.99 and 2.00, and the forward value approaches 1 further as h decreases (0.994 between 5×10^{-3} and 2.5×10^{-3}). It is slightly below 1 because the next term of the Taylor series, $\frac{h^2}{6}u'''$, has the opposite sign of the leading term $\frac{h}{2}u''$ ($u'' < 0$, $u''' > 0$) and reduces the error at the larger step a little more.

That round-off plays no role at $h = 10^{-2}$ follows from the two terms of the error bound in the script, evaluated with the numbers above and $\varepsilon = 2.2 \times 10^{-16}$:

$$\frac{h}{2}|u''| = \frac{10^{-2}}{2} \cdot 0.7788 = 3.9 \times 10^{-3}, \qquad \frac{2\varepsilon|u|}{h} = \frac{2 \cdot 2.2 \times 10^{-16} \cdot 0.7788}{10^{-2}} = 3.4 \times 10^{-14}.$$

The round-off part is eleven orders of magnitude smaller than the truncation part. For the central difference the comparison is $\frac{h^2}{6}|u'''| = 6.5 \times 10^{-5}$ against $\varepsilon|u|/h = 1.7 \times 10^{-14}$.

c) We insert the values of $|u|$, $|u''|$ and $|u'''|$ into the estimates of the script. For the forward difference $|u| = |u''| = 0.7788$, so the ratio under the root is one:

$$h_{\text{opt}} = 2\sqrt{\frac{\varepsilon|u|}{|u''|}} = 2\sqrt{\varepsilon \cdot 1} = 2 \cdot 1.49 \times 10^{-8} = 3.0 \times 10^{-8},$$

$$e(h_{\text{opt}}) = 2\sqrt{\varepsilon\,|u|\,|u''|} = 2\sqrt{\varepsilon} \cdot \sqrt{0.7788 \cdot 0.7788} = 2 \cdot 1.49 \times 10^{-8} \cdot 0.7788 = 2.3 \times 10^{-8}.$$

For the central difference $|u|/|u'''| = 0.7788/3.894 = 0.2$, so

$$h_{\text{opt}} = \left(\frac{3\varepsilon|u|}{|u'''|}\right)^{1/3} = \left(3 \cdot 2.22 \times 10^{-16} \cdot 0.2\right)^{1/3} = \left(1.33 \times 10^{-16}\right)^{1/3} = 5.1 \times 10^{-6},$$

$$e(h_{\text{opt}}) = \frac{h_{\text{opt}}^2}{2}|u'''| = \frac{(5.1 \times 10^{-6})^2}{2} \cdot 3.894 = \frac{2.6 \times 10^{-11}}{2} \cdot 3.894 = 5.1 \times 10^{-11}.$$

The code prints the same values. In the plot, the cloud of points stops falling near these steps, at about these errors: near $h = 10^{-8}$ with errors of about 10^{-8}, and near $h = 10^{-5}$ with errors

of about 10^{-11}. The single lowest point, which the code finds, lies lower and at a somewhat different h (2.7×10^{-10} at $h = 1.8 \times 10^{-9}$ and 2.4×10^{-12} at $h = 3.2 \times 10^{-6}$). This is luck: at some values of h the rounding errors δ_1 and δ_2 happen to cancel. The estimates are upper bounds; they describe the typical error, that is the upper edge of the cloud, and this is what should be read from the plot. Left of the optimum the error grows as ε/h and scatters, because round-off errors are irregular.

Points: a) 2, b) 1, c) 1.

Problem 2: A one-sided difference (4 points)

At a boundary at x only the points x, $x + h$ and $x + 2h$ are available.

a) Use the Taylor series to find α, β, γ such that $[\alpha\, u(x) + \beta\, u(x + h) + \gamma\, u(x + 2h)]\, /h$ approximates $u'(x)$ with the highest possible order. Give the leading term of the truncation error.

b) Confirm the order and the leading error term with a convergence test for a function of your choice.

Solution

a) The Taylor series of the two neighbours, with u, u', ... evaluated at x, are

$$\begin{aligned} u(x+h) &= u + hu' + \tfrac{1}{2}h^2u'' + \tfrac{1}{6}h^3u''' + \dots \\ u(x+2h) &= u + (2h)u' + \tfrac{1}{2}(2h)^2u'' + \tfrac{1}{6}(2h)^3u''' + \dots \\ &= u + 2hu' + \tfrac{4}{2}h^2u'' + \tfrac{8}{6}h^3u''' + \cdots = u + 2hu' + 2h^2u'' + \tfrac{4}{3}h^3u''' + \dots \end{aligned}$$

We multiply the first line by β, the second by γ, add αu and collect the terms with the same derivative:

$$\begin{aligned} \alpha u(x) + \beta u(x+h) + \gamma u(x+2h) = (\alpha+\beta+\gamma)\, u + (\beta+2\gamma)\, hu' + \left(\tfrac{1}{2}\beta + 2\gamma\right) h^2u'' \\ + \left(\tfrac{1}{6}\beta + \tfrac{4}{3}\gamma\right) h^3u''' + \dots \end{aligned}$$

Divided by h, the right hand side reads $(\alpha+\beta+\gamma)\, u/h + (\beta+2\gamma)\, u' + (\frac{1}{2}\beta+2\gamma)\, hu'' + \dots$. This shall be u' plus an error that is as small as possible. With three coefficients we can impose three conditions: the coefficient of u vanishes (otherwise the result would grow as $1/h$), that of u' is one, and that of hu'' vanishes:

$$\begin{aligned} \alpha + \beta + \gamma &= 0, \\ \beta + 2\gamma &= 1, \\ \tfrac{1}{2}\beta + 2\gamma &= 0. \end{aligned}$$

Subtracting the third equation from the second removes γ: $\beta - \frac{1}{2}\beta = 1 - 0$, that is $\frac{1}{2}\beta = 1$ and $\beta = 2$. The third equation then gives $1 + 2\gamma = 0$, so $\gamma = -\frac{1}{2}$, and the first $\alpha = -\beta - \gamma = -2 + \frac{1}{2} = -\frac{3}{2}$.

The first term that is not cancelled is the $h^3 u'''$ term, with the coefficient

$$\tfrac{1}{6}\beta + \tfrac{4}{3}\gamma = \tfrac{1}{6} \cdot 2 + \tfrac{4}{3} \cdot \left(-\tfrac{1}{2}\right) = \tfrac{1}{3} - \tfrac{2}{3} = -\tfrac{1}{3}.$$

After division by h,

$$\frac{-\frac{3}{2}u(x) + 2u(x+h) - \frac{1}{2}u(x+2h)}{h} = u'(x) - \frac{h^2}{3}u'''(x) + \dots$$

We multiply numerator and denominator on the left by 2, which removes the fractions, and bring the error term to the other side; its sign changes from $-$ to $+$:

$$u'(x) = \frac{-3u(x) + 4u(x+h) - u(x+2h)}{2h} + \frac{h^2}{3}u'''(x) + \dots$$

The formula is of second order with the leading error term $\frac{h^2}{3}u'''(x)$. Its error constant, $1/3$, is twice that of the central difference, $1/6$. Two checks: the weights $-3 + 4 - 1$ add up to zero, so a constant u gives the derivative zero; and for $u = x$ (take $x = 0$) the formula gives $(0 + 4h - 2h)/(2h) = 1$, as it should.

b) For the convergence test we take $u = \sin$ at $x = 1$, so $u' = \cos$ and $u''' = -\cos$. From a), the *signed* error of the formula is

$$D_h - u'(x) = -\frac{h^2}{3}u'''(x) + \dots = -\frac{h^2}{3}\big(-\cos 1\big) + \dots = +\frac{h^2}{3}\cos 1 + \dots,$$

where D_h is the difference quotient. The two minus signs cancel, so the error must be positive, and $(D_h - u')/h^2$ must approach $\cos(1)/3 = 0.5403/3 = 0.1801$. The code prints the signed error, so that it also checks the sign.

Step by step

```
x = 1.0
h_list = [0.1, 0.05, 0.025, 0.0125]
errors = []
for h in h_list:
    onesided = (-3 * np.sin(x) + 4 * np.sin(x + h) - np.sin(x + 2 * h)) / (2 * h)
    error = onesided - np.cos(x)    # signed error, should be > 0
    errors.append(error)
    print(f"h = {h:6.4f}: error = {error:.2e}, error / h^2 = {error / h**2:.4f}")

for i in range(1, len(h_list)):
    p = np.log(errors[i - 1] / errors[i]) / np.log(h_list[i - 1] / h_list[i])
    print(f"observed order between h = {h_list[i - 1]} and {h_list[i]}: {p:.3f}")
print("expected error / h^2 for small h: cos(1) / 3 =", round(np.cos(x) / 3, 4))
```

```
h = 0.1000: error = 1.58e-03, error / h^2 = 0.1585
h = 0.0500: error = 4.24e-04, error / h^2 = 0.1694
h = 0.0250: error = 1.09e-04, error / h^2 = 0.1748
h = 0.0125: error = 2.77e-05, error / h^2 = 0.1775
observed order between h = 0.1 and 0.05: 1.904
observed order between h = 0.05 and 0.025: 1.955
observed order between h = 0.025 and 0.0125: 1.978
expected error / h^2 for small h: cos(1) / 3 = 0.1801
```

Compact

```
x = 1.0
h = np.array([0.1, 0.05, 0.025, 0.0125])    # each h is half the previous one
onesided = (-3 * np.sin(x) + 4 * np.sin(x + h) - np.sin(x + 2 * h)) / (2 * h)
error = onesided - np.cos(x)
print("observed order:", np.log2(error[:-1] / error[1:]).round(3))
print("error / h^2:   ", (error / h**2).round(4))
```

```
observed order: [1.904 1.955 1.978]
error / h^2:    [0.1585 0.1694 0.1748 0.1775]
```

The errors are positive, as predicted. The observed order tends to 2, and error/h^2 approaches 0.1801 from below, which confirms also the constant and the sign of the leading error term. That the approach is slow (0.1775 at $h = 0.0125$) is due to the next term of the series, of order h^3.

Points: a) 3, b) 1.

Problem 3: Quadrature (2 points)

Implement the composite trapezoidal rule and Simpson's rule. Compute $\int_0^\pi \sin x \, \mathrm{d}x = 2$ with $N = 4, 8, 16, 32$ subintervals and determine the observed order of both rules.

Solution

The rules. With $h = (b-a)/N$ and the points $x_i = a + ih$, the composite trapezoidal rule is

$$T_N = h\left[\tfrac{1}{2}f(x_0) + f(x_1) + \cdots + f(x_{N-1}) + \tfrac{1}{2}f(x_N)\right].$$

For Simpson's rule we use the form of the script. On one subinterval with midpoint m,

$$\tfrac{2}{3}\cdot \underbrace{hf(m)}_{\text{midpoint}} + \tfrac{1}{3}\cdot \underbrace{\tfrac{h}{2}\left[f(m-\tfrac{h}{2}) + f(m+\tfrac{h}{2})\right]}_{\text{trapezoidal}} = \frac{h}{6}\left[f(m-\tfrac{h}{2}) + 4f(m) + f(m+\tfrac{h}{2})\right].$$

Summing over all subintervals, the left hand side becomes $\frac{2}{3}$ of the composite midpoint rule plus $\frac{1}{3}$ of the composite trapezoidal rule:

$$S_N = \tfrac{2}{3}M_N + \tfrac{1}{3}T_N = \frac{2M_N + T_N}{3}, \qquad M_N = h\sum_{i=0}^{N-1} f\Big(a + (i+\tfrac{1}{2})h\Big).$$

The observed order. Since N doubles from one row to the next, h halves, $h_1/h_2 = 2$, and the formula of Problem 1 becomes $p = \ln(e_N/e_{2N})/\ln 2 = \log_2(e_N/e_{2N})$.

A sign check. The script gives the error of the composite trapezoidal rule as $\int f - T_N = -\frac{h^2}{12}[f'(b) - f'(a)] + \ldots$. For $f = \sin$ on $[0, \pi]$, $f' = \cos$ and

$$f'(\pi) - f'(0) = \cos\pi - \cos 0 = -1 - 1 = -2, \qquad 2 - T_N = -\frac{h^2}{12}\cdot(-2) + \cdots = +\frac{h^2}{6} + \ldots$$

The error is positive: the trapezoidal rule underestimates the integral of the concave sine, because the chords lie below the arc. The code prints $(2 - T_N)/h^2$, which should approach $1/6 = 0.1667$.

Step by step

```python
def trapezoidal(f, a, b, n):
    h = (b - a) / n                 # width of a subinterval
    total = 0.5 * f(a) + 0.5 * f(b) # the end points count half
    for i in range(1, n):           # interior points i = 1, ..., n-1
        total = total + f(a + i * h)
    return h * total

def midpoint(f, a, b, n):
    h = (b - a) / n
    total = 0.0
    for i in range(n):              # midpoints of the subintervals 0, ..., n-1
        total = total + f(a + (i + 0.5) * h)
    return h * total

def simpson(f, a, b, n):
    return (2 * midpoint(f, a, b, n) + trapezoidal(f, a, b, n)) / 3

n_list = [4, 8, 16, 32]
error_t = []
error_s = []
for n in n_list:
    h = np.pi / n
    e_t = 2 - trapezoidal(np.sin, 0, np.pi, n)   # signed: exact - rule
    e_s = 2 - simpson(np.sin, 0, np.pi, n)
    error_t.append(abs(e_t))
    error_s.append(abs(e_s))
    print(f"N = {n:2d}: (2 - T) / h^2 = {e_t / h**2:.4f}")

print(" N   error trapezoidal  order   error Simpson  order")
for i in range(len(n_list)):
    if i == 0:                      # no order for the first row
        print(f"{n_list[i]:2d}   {error_t[i]:12.2e}           {error_s[i]:12.2e}")
    else:
        p_t = np.log2(error_t[i - 1] / error_t[i])
        p_s = np.log2(error_s[i - 1] / error_s[i])
        print(f"{n_list[i]:2d}   {error_t[i]:12.2e}  {p_t:6.2f}"
              f"   {error_s[i]:12.2e}  {p_s:6.2f}")
```

```
N =  4: (2 - T) / h^2 = 0.1684
N =  8: (2 - T) / h^2 = 0.1671
N = 16: (2 - T) / h^2 = 0.1668
N = 32: (2 - T) / h^2 = 0.1667
 N   error trapezoidal  order   error Simpson  order
 4       1.04e-01               2.69e-04
 8       2.58e-02    2.01       1.66e-05    4.02
16       6.43e-03    2.00       1.03e-06    4.00
32       1.61e-03    2.00       6.45e-08    4.00
```

Compact

```
def trapezoidal(f, a, b, n):
    x = np.linspace(a, b, n + 1)            # the n + 1 points x_0, ..., x_n
    h = (b - a) / n
    return h * (0.5 * f(x[0]) + np.sum(f(x[1:-1])) + 0.5 * f(x[-1]))

def simpson(f, a, b, n):
    h = (b - a) / n
    midpoints = a + h * (np.arange(n) + 0.5)
    return (2 * h * np.sum(f(midpoints)) + trapezoidal(f, a, b, n)) / 3

n = np.array([4, 8, 16, 32])
error_t = np.zeros(len(n))
error_s = np.zeros(len(n))
for i in range(len(n)):
    error_t[i] = abs(trapezoidal(np.sin, 0, np.pi, n[i]) - 2)
    error_s[i] = abs(simpson(np.sin, 0, np.pi, n[i]) - 2)
print("observed order, trapezoidal:", np.log2(error_t[:-1] / error_t[1:]).round(2))
print("observed order, Simpson:    ", np.log2(error_s[:-1] / error_s[1:]).round(2))
```

```
observed order, trapezoidal: [2.01 2.   2.  ]
observed order, Simpson:     [4.02 4.   4.  ]
```

The trapezoidal rule is of second order, Simpson's rule of fourth order: the errors fall by factors of 4 and 16 when N is doubled. The ratio $(2 - T_N)/h^2$ is positive and approaches $1/6$, as predicted by the sign check.

Points: 2.

Problem 4: Root finding (3 points)

How deep is the water in a river that carries a given discharge? For a channel with a rectangular cross section of width b, Manning's formula gives the discharge at the water depth y:

$$Q(y) = \frac{\sqrt{S}}{n} A R^{2/3}, \qquad A = b\,y, \qquad R = \frac{A}{b + 2y}.$$

A is the area of the cross section, and the hydraulic radius R is this area divided by the wetted perimeter $b + 2y$ (bottom and both banks). Take the width $b = 10$ m, the bed slope $S = 0.001$ and the roughness $n = 0.035$ s m$^{-1/3}$ of a natural river bed. We seek the depth at which the river carries $Q_0 = 50$ m^3 s^{-1}, the root of $f(y) = Q(y) - Q_0 = 0$.

a) Write two functions of your own, `bisection(f, a, b, tol)` and `newton(f, dfdx, x0, tol)`. Each returns the root and the number of iterations it needed.

b) The derivative is

$$f'(y) = Q(y)\left(\frac{5}{3y} - \frac{4}{3(b+2y)}\right).$$

Check it with a central difference before you use it. Find the depth with bisection and with Newton's method to a tolerance of 10^{-12} m. Justify your bracket and starting value with a plot. Compare the number of iterations.

c) Start Newton's method at $y_0 = 0.01$ m. What happens, and why? Why can it not start at $y_0 = 0$?

Solution

a) **Bisection** keeps an interval $[a, b]$ in which f changes sign. It evaluates f at the midpoint $c = (a + b)/2$ and keeps the half in which the sign change lies: $[a, c]$ if $f(a)f(c) \le 0$, otherwise $[c, b]$. It stops when half the interval length, the largest possible distance of the midpoint from the root, is below the tolerance.

Newton's method replaces f near the current estimate y_k by its tangent, the first two terms of the Taylor series, $f(y_k + \delta) \approx f(y_k) + \delta\, f'(y_k)$, and sets this to zero: $\delta = -f(y_k)/f'(y_k)$. Hence

$$y_{k+1} = y_k - \frac{f(y_k)}{f'(y_k)}.$$

It stops when the step $|\delta|$ is below the tolerance; a maximum number of iterations prevents an endless loop if it does not converge.

b) **Where the derivative comes from** (not required). Inserting $R = A/P$ with the wetted perimeter $P = b + 2y$ gives $AR^{2/3} = A \cdot A^{2/3}P^{-2/3}$, so

$$Q(y) = \frac{\sqrt{S}}{n}\, A^{5/3}P^{-2/3}, \qquad \frac{\mathrm{d}A}{\mathrm{d}y} = b, \qquad \frac{\mathrm{d}P}{\mathrm{d}y} = 2.$$

The product rule and the chain rule give

$$Q'(y) = \frac{\sqrt{S}}{n}\left(\frac{5}{3}A^{2/3}\, b\, P^{-2/3} - \frac{2}{3}A^{5/3}P^{-5/3} \cdot 2\right).$$

Factoring out $\frac{\sqrt{S}}{n}A^{5/3}P^{-2/3} = Q(y)$ leaves $\frac{5}{3}\frac{b}{A} - \frac{4}{3}\frac{1}{P}$, and with $b/A = 1/y$,

$$f'(y) = Q'(y) = Q(y)\left(\frac{5}{3y} - \frac{4}{3(b+2y)}\right).$$

Q_0 is a constant and drops out. The expression in parentheses is positive for $y > 0$, because $3y < 3(b+2y)$ and $5 > 4$: the discharge grows with the depth, so there is only one root. The code compares $f'(2)$ with a central difference, because a wrong derivative, for example a typo in the code, makes Newton's method converge slowly or run away.

Step by step

```
width = 10.0                          # b in the formula, m
S = 0.001                             # bed slope
n = 0.035                             # Manning roughness, s m^(-1/3)
Q0 = 50.0                             # discharge we want, m^3/s

def Q(y):                             # Manning's formula
    A = width * y                     # area of the cross section
    P = width + 2 * y                 # wetted perimeter
    R = A / P                         # hydraulic radius
    return np.sqrt(S) / n * A * R**(2 / 3)

def f(y):
    return Q(y) - Q0

def dfdy(y):                          # derivative of f, derived above
    return Q(y) * (5 / (3 * y) - 4 / (3 * (width + 2 * y)))

# check of the derivative with a central difference at y = 2
h = 1e-5
print("f'(2) analytic:", dfdy(2.0), " central difference:",
      (f(2.0 + h) - f(2.0 - h)) / (2 * h))

def bisection(f, a, b, tol):
    fa = f(a)
    k = 0                             # counts the iterations
    while (b - a) / 2 > tol:
        c = (a + b) / 2               # midpoint
        fc = f(c)
        k = k + 1
        if fa * fc <= 0:              # sign change in [a, c]
            b = c
        else:                         # sign change in [c, b]
            a = c
            fa = fc
    return (a + b) / 2, k
```

```
def newton(f, dfdx, x, tol):
    k = 0
    step = 2 * tol                  # any value > tol, so that the loop starts
    while abs(step) > tol and k < 100:
        step = -f(x) / dfdx(x)
        x = x + step
        k = k + 1
    return x, k

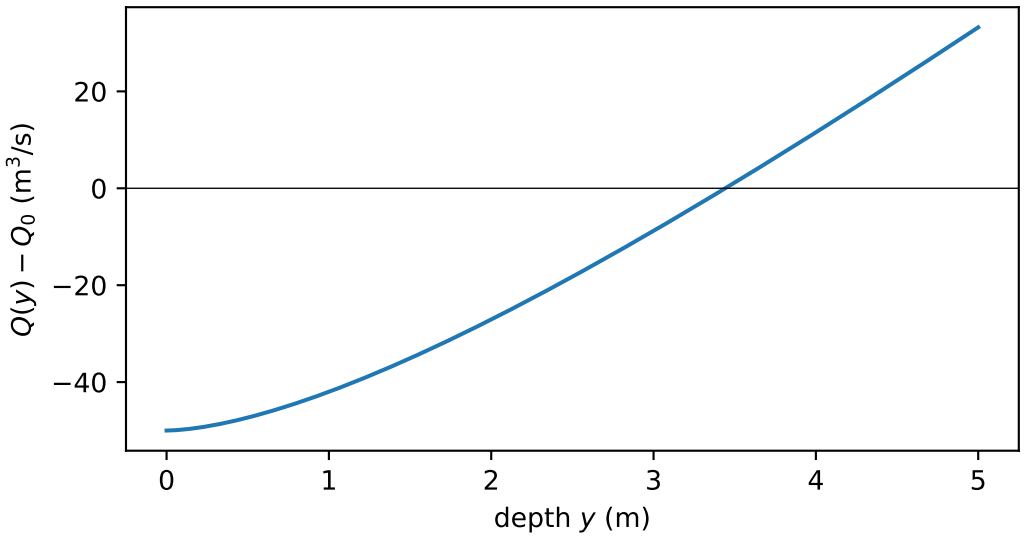
# plot to find the bracket and the starting value
y_values = []
f_values = []
for i in range(501):
    y = i * 5 / 500                 # 501 depths from 0 to 5 m
    y_values.append(y)
    f_values.append(f(y))
fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(y_values, f_values)
ax.axhline(0, color="k", lw=0.5)    # horizontal line at zero
ax.set_xlabel("depth $y$ (m)")
ax.set_ylabel(r"$Q(y) - Q_0$ (m$^3$/s)")
plt.show()

# b) bisection on [0, 5] m, Newton from the midpoint
root_b, k_b = bisection(f, 0.0, 5.0, 1e-12)
root_n, k_n = newton(f, dfdy, 2.5, 1e-12)
print(f"bisection {root_b:.10f} m ({k_b} iterations),"
      f" Newton {root_n:.10f} m ({k_n} iterations)")

# b) the Newton steps once more, to see the quadratic convergence
y = 2.5
for k in range(4):
    step = -f(y) / dfdy(y)
    y = y + step
    print(f"iteration {k + 1}: step {abs(step):.1e}")

# c) start at 0.01 m, printing every iterate
y = 0.01
print(f"f(0.01) = {f(y):.3f}, f'(0.01) = {dfdy(y):.3f}")
for k in range(7):
    y = y - f(y) / dfdy(y)
    print(f"iteration {k + 1}: y = {y:.6f} m")
```

```
f'(2) analytic: 16.917739292833176  central difference: 16.917739292843237
```




```
bisection 3.4420198928 m (42 iterations), Newton 3.4420198928 m (5 iterations)
iteration 1: step 9.9e-01
iteration 2: step 5.3e-02
iteration 3: step 1.1e-04
iteration 4: step 5.0e-10
f(0.01) = -49.996, f'(0.01) = 0.697
iteration 1: y = 71.692248 m
iteration 2: y = 4.883366 m
iteration 3: y = 3.496070 m
iteration 4: y = 3.442136 m
iteration 5: y = 3.442020 m
iteration 6: y = 3.442020 m
iteration 7: y = 3.442020 m
```

Compact

```python
from scipy.optimize import root_scalar

width, S, n, Q0 = 10.0, 0.001, 0.035, 50.0
Q = lambda y: np.sqrt(S) / n * width * y * (width * y / (width + 2 * y))**(2 / 3)
f = lambda y: Q(y) - Q0
dfdy = lambda y: Q(y) * (5 / (3 * y) - 4 / (3 * (width + 2 * y)))

rb = root_scalar(f, bracket=[0.0, 5.0], method="bisect", xtol=1e-12)
rn = root_scalar(f, fprime=dfdy, x0=2.5, method="newton", xtol=1e-12)
print(f"bisection {rb.root:.10f} m ({rb.iterations} iterations),"
```

```
      f" Newton {rn.root:.10f} m ({rn.iterations} iterations)")

rn = root_scalar(f, fprime=dfdy, x0=0.01, method="newton", xtol=1e-12)
print(f"start at 0.01 m: {rn.root:.10f} m after {rn.iterations} iterations")
```

```
bisection 3.4420198928 m (43 iterations), Newton 3.4420198928 m (5 iterations)
start at 0.01 m: 3.4420198928 m after 6 iterations
```

The analytic derivative and the central difference agree to ten digits.

Bracket. Without water there is no discharge: $Q(0) = 0$, so $f(0) = -50 < 0$. At $y = 5$ m, $A = 50$ m^2, $P = 20$ m, $R = 2.5$ m and $Q = 0.9035 \cdot 50 \cdot 2.5^{2/3} = 83.2$ m^3 s^{-1}, so $f(5) = 33.2 > 0$. f is continuous on $[0, 5]$ and changes sign, so the root lies in between, as the plot confirms. The midpoint 2.5 m is a reasonable starting value for Newton's method.

Iterations of bisection. The interval has the length $L = 5$ m. After k halvings its length is $L/2^k$, and the loop stops when half of it is at most the tolerance:

$$\frac{L}{2^{k+1}} \le 10^{-12} \quad \Longleftrightarrow \quad 2^{k+1} \ge \frac{L}{10^{-12}} \quad \Longleftrightarrow \quad k \ge \log_2\left(5 \cdot 10^{12}\right) - 1.$$

With $\log_2 5 = 2.32$ and $\log_2 10^{12} = 12 \log_2 10 = 12 \cdot 3.32 = 39.86$, the right hand side is $2.32 + 39.86 - 1 = 41.2$, so bisection needs $k = 42$ iterations, as the code finds. (SciPy counts 43, because its stopping test differs slightly.)

The depth is $y = 3.4420$ m. As a check: the mean velocity is $Q_0/A = 50/34.4 = 1.45$ m s^{-1}, a plausible value for a river.

Iterations of Newton's method. Five iterations suffice, the last one only confirming that the step is below the tolerance. In the script we derived $e_{k+1} \approx \frac{f''(r)}{2f'(r)}\, e_k^2$ for the error $e_k = y_k - r$: the number of correct digits roughly doubles in each iteration. The printed steps, which approximate the errors $|e_k|$, show this: 9.9×10^{-1}, 5.3×10^{-2}, 1.1×10^{-4}, 5.0×10^{-10}; the exponent roughly doubles from step to step. Bisection gains only one binary digit, a factor of 2, per iteration.

c) At $y_0 = 0.01$ m the river carries almost nothing and the curve is nearly flat:

$$f(0.01) = -49.996, \qquad f'(0.01) = 0.697.$$

The tangent is almost horizontal and crosses zero far away: the step is

$$\delta = -\frac{f(0.01)}{f'(0.01)} = \frac{49.996}{0.697} = 71.7,$$

which throws the iterate to $y_1 = 71.7$ m, a water depth that has nothing to do with the river. Here the method is lucky and recovers: for $y \gg b$ the hydraulic radius approaches $b/2$, so Q grows almost linearly with y, and for a nearly linear function the tangent is a good approximation. The next iterates are 4.88, 3.50, 3.4421 m, and after six iterations the root is found to full accuracy.

The reason for the large first step is that $f'(y) \to 0$ as $y \to 0$: for small y, $P \approx b$ and $Q \approx \frac{\sqrt{S}}{n} b\, y^{5/3} b^{-2/3}$, whose derivative is proportional to $y^{2/3}$. At $y_0 = 0$ the tangent is exactly horizontal and never crosses zero; the Newton step $-f(0)/f'(0) = 50/0$ is not defined. (The formula for f' above gives $0 \cdot \infty$ there, and Python stops with a `ZeroDivisionError`.) Newton's method needs a starting value in the neighbourhood of the root, and where $f' \approx 0$ the tangent throws it far away; a few bisection steps provide a safe start.

Points: a) 1, b) 1, c) 1.